

Bitácora

Sprint 1: Planificación e infraestructura

Primero investigue las tecnologías que usar. Me decanté por Spring Boot porque tanto en el ciclo como en las prácticas de Ayesa trabajo con él. Para el frontend elegí React porque es el framework que vi en clases y con el que me siento más agusto. Y usaré PostgreSQL como base de datos ya que la uso en la empresa, vi que se pide mucho en las empresas y por los CHECK constraints.

Al principio comencé desarrollando después de planear, pero después de unas cuantas clases pare ya que no tenía claro el flujo completo. Por lo que empecé a diseñar los wireframes de las pantallas principales: login, registro, home, listado de usuarios y votaciones. Los hice sin colores para centrarme en la estructura y los flujos de navegación. Esto me llevó bastante tiempo ya que me hizo plantearme toda la aplicación desde 0, viendo el flujo de un vecino y un presidente, haciendo que tuviera que reformular muchas decisiones que tenía pero no encajaban en el flujo de trabajo. Gracias a esta tarea puedo ver la aplicación de principio a fin haciendo que desarrollar sea mucho más rápido.

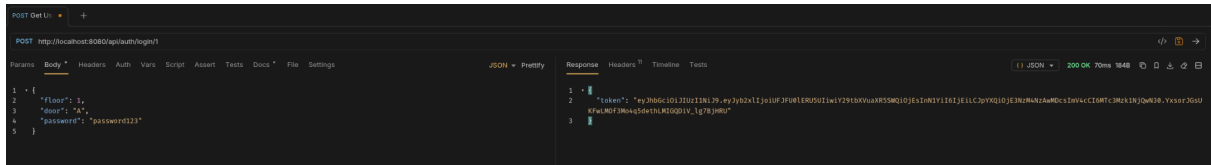
Diseñé el esquema de base de datos. Empecé con las tablas básicas y fui añadiendo las relaciones. Este apartado me hizo pensar nuevamente en el funcionamiento de toda la aplicación y tomar decisiones según los objetivos definidos.

Configuré toda la infraestructura: creé el repositorio en GitHub, las ramas main y develop, el docker-compose.yml para PostgreSQL, la estructura de carpetas del proyecto y los archivos .env. Probé que Docker levantaba bien la base de datos y ejecutaba los scripts de init.

Sprint 2: Login

Empecé con la autenticación del backend. Creé el LoginRequest, LoginResponse, el AuthService con la lógica de buscar usuario por planta y puerta, y la verificación de contraseña con BCrypt. Configuré Spring Security para que todos los endpoints fueran públicos de momento.

Implementé el JwtService para generar tokens. Creé el AuthController con el endpoint POST /api/auth/login/{communityId}. Probé con Bruno que el login devolvía un JWT válido y que las credenciales incorrectas devolvían 401.



Después seguí con el frontend. Configuré React Router con las rutas de login y home, e implementé el AuthContext con el hook useAuth para gestionar el token en localStorage. Creé el servicio de Axios con la baseURL apuntando al backend.

Terminé el formulario de login con React Hook Form y validación con Yup. Conecté el formulario con el backend a través del authService. Creé el ProtectedRoute para que la home solo sea accesible con token. Tuve un problema con CORS que resolví configurando los orígenes permitidos en Spring Security.

Creé el CSS del login para que se viese limpio y centrado. Añadí mi icono para mayor personalización.

Hice unas pruebas finales del flujo completo: login correcto redirige al home, login incorrecto muestra error, acceder al home sin token redirige al login.

Y para finalizar terminé la documentación completa e hice mi primera subida a Github.